NILM Project - Milestone 2

Modeling, Simulation and Automation of Electrical Energy Systems

Team	Soheil Ayati (11153003), Marc Steffgen (11149043)
Milestone	2 - Feature extraction, modelling, training, and real-meter measurements
Repository	https://github.com/SoheilAyati/MSAEES

1. Objectives

Milestone 1 delivered the data side of the project: synthetic appliance generators, a scenario aggregator that mimics a Siemens PAC4200 at the point of common coupling, a universal preprocessor, and a Modbus reader for the real meter. Milestone 2 delivers the analysis half: turning a signal into per-appliance answers with machine learning, and validating the chain on real measurements taken from the lab PAC4200.
The pipeline answers three questions, each a separate task, and supports three model families so a classical and a neural baseline can be compared on the same data:

- **Identify -** which single appliance is this? (one label per active window).
- **Disaggregate -** how much power does each appliance draw? (watts per appliance, per window).
- **Presence -** which appliances are ON right now? (a multi-label on/off timeline).

2. Pipeline architecture

A single shared library, *nilm_pipeline.py*, loads any input source. For example a real PAC4200 measurement, a single-appliance synthetic file, or an aggregate scenario is loaded into one uniform Signal object, so the rest of the code never special-cases the source. *train.py* learns a model and reports held-out metrics; *infer.py* runs a saved model on one signal and renders result plots; deep_models.py is the neural-network path; *generate_corpus.py* builds synthetic corpora; and app.py wraps all of it in a point-and-click interface.

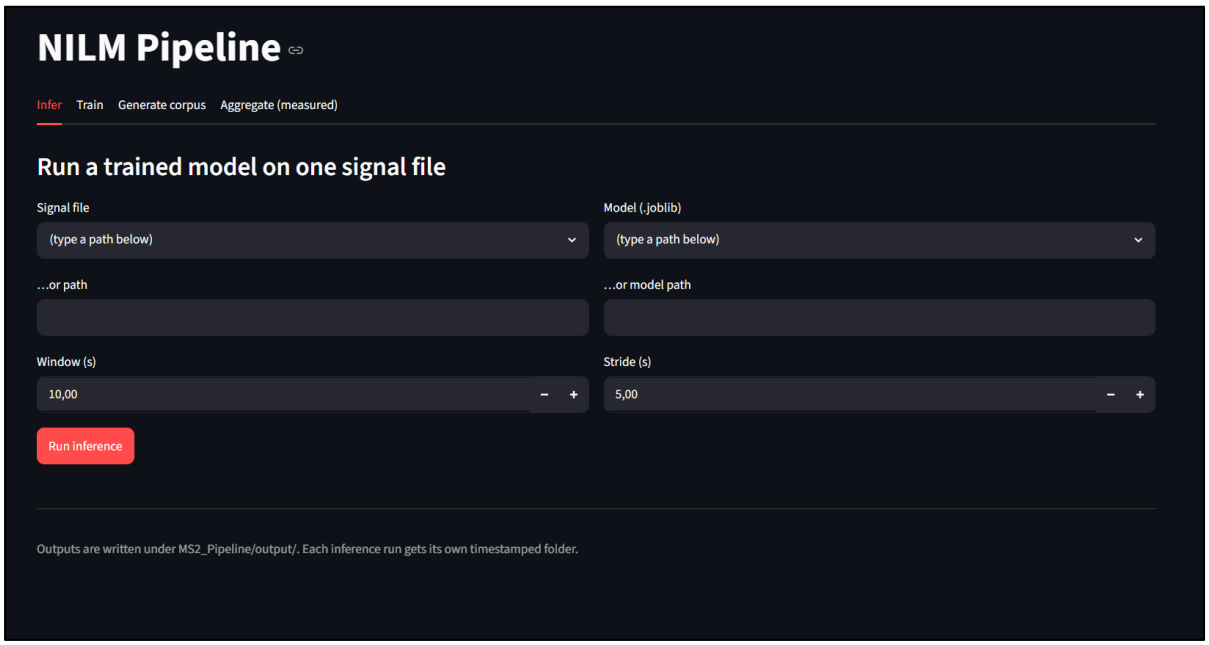

Figure 1. *The Streamlit control panel - one UI for inference, training, corpus generation, and aggregating real recordings.*

Features and windowing

The 5 Hz signal is cut into fixed windows (length and stride in seconds; a window is “active” when its mean |*P*| exceeds an on-threshold). Identification uses a compact common feature set (*P*, *Q*, *S*, *power factor*, *Q/P ratio*, *THD_I* and *P statistics*) that a real PAC4200 measurement also provides, so a model trained on synthetic data can transfer to the meter; a fuller set adds per-order harmonic features for synthetic files. *THD_I* is measured where available and derived from the harmonic spectrum otherwise, keeping features comparable across sources. Disaggregation and presence use aggregate, per-phase window features, while the neural path learns from the raw windowed waveform.

3. Key design decisions

- **Source-agnostic loading.** A file is treated as a disaggregation scenario only when it actually carries per-appliance ground truth and not from its channel names. This is what lets real single appliance recordings (which share the aggregate channel layout but have no ground truth) load correctly instead of being mistaken for scenarios.
- **Honest evaluation.** When several instances of an appliance exist, the held-out split keeps whole instances apart (GroupShuffleSplit), so the score reflects generalisation rather than memorisation; with a single instance it falls back to a stratified split and says so.
- **Real-meter compatibility.** The common feature subset is exactly what the PAC4200 exposes, so identification transfers from synthetic training to the real device without changing features.
- **Reuse, don’t reinvent.** Real measurements are folded into the existing Milestone 1 aggregator rather than a parallel code path, so synthetic and measured mixtures share identical physics and format.

4. From synthetic to measured data

Single appliances were recorded with the lab PAC4200 through the Milestone-1 reader: *a laptop, three stand coolers, table fans at several speeds, and a PV panel*. Each saved as its own file. The loader script was extended to accept these recordings: real meters do not expose *current-THD* by default, so a missing *THD_I* channel now falls back gracefully, and the appliance label is read from the recording metadata.

To build mixed scenarios from real data, *mix_measured_scenarios.py* converts each recording into the per-appliance format, loops it to a common length, and calls the unchanged Milestone 1 aggregator to synthesise aggregate scenarios that carry exact per-appliance ground truth. This is exposed as an “Aggregate (measured)” tab in the web app. The appliance list was extended with the measured families, so the disaggregation and presence targets are built from the real appliances rather than collapsing to zero.

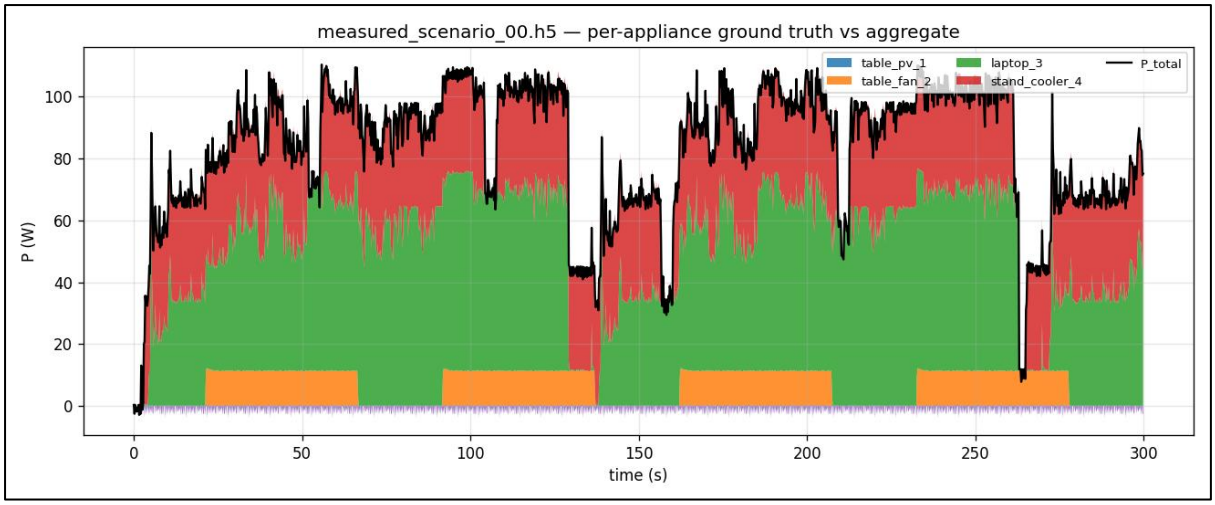

Figure 2. *A scenario mixed from real recordings: the stacked per-appliance ground truth sums exactly to the aggregate the model sees.*

5. Results

Identification was trained on the live single-appliance recordings using the real-meter compatible feature set. A first model reached about 0.91 accuracy. Given that each appliance was recorded only once, this is an optimistic within-recording estimate that repeated recording sessions will tighten.

Presence was then trained on five mixed measured scenarios and tested on a held-out sixth. The appliances above the 15 W presence threshold (the laptop and the stand cooler) were recovered with per-appliance 1.0 accuracy, matching the ground-truth timeline exactly; the low-power table fan (~10-14 W) and the near-zero PV clip fall below the threshold and are reported absent.

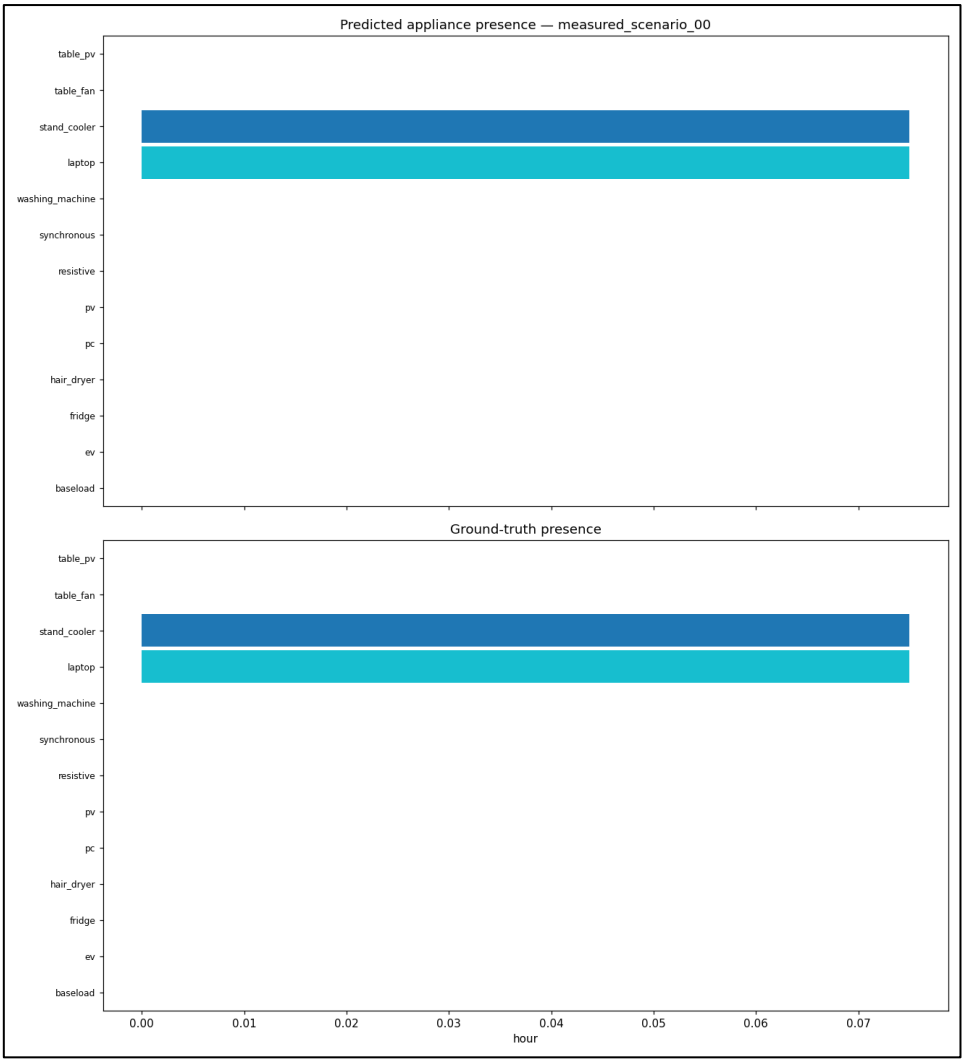

Figure 3. *Presence on a held-out measured scenario - predicted (top) vs ground-truth (bottom) appliance activity.*

Presence was also evaluated on the fully labelled synthetic data. Training on the scenarios and testing on a held-out scenario (scenario_seed115, unseen in training), the Random-Forest model recovered the nine synthetic appliances with strong per-appliance accuracy and lower accuracy for the washing machine, whose low-power agitate/rinse phases are buried under other loads and exceed a single 30s window. This shows the same pipeline that flags the real laptop and stand cooler scales to a full nine-appliance mixture when richer labelled data is available.

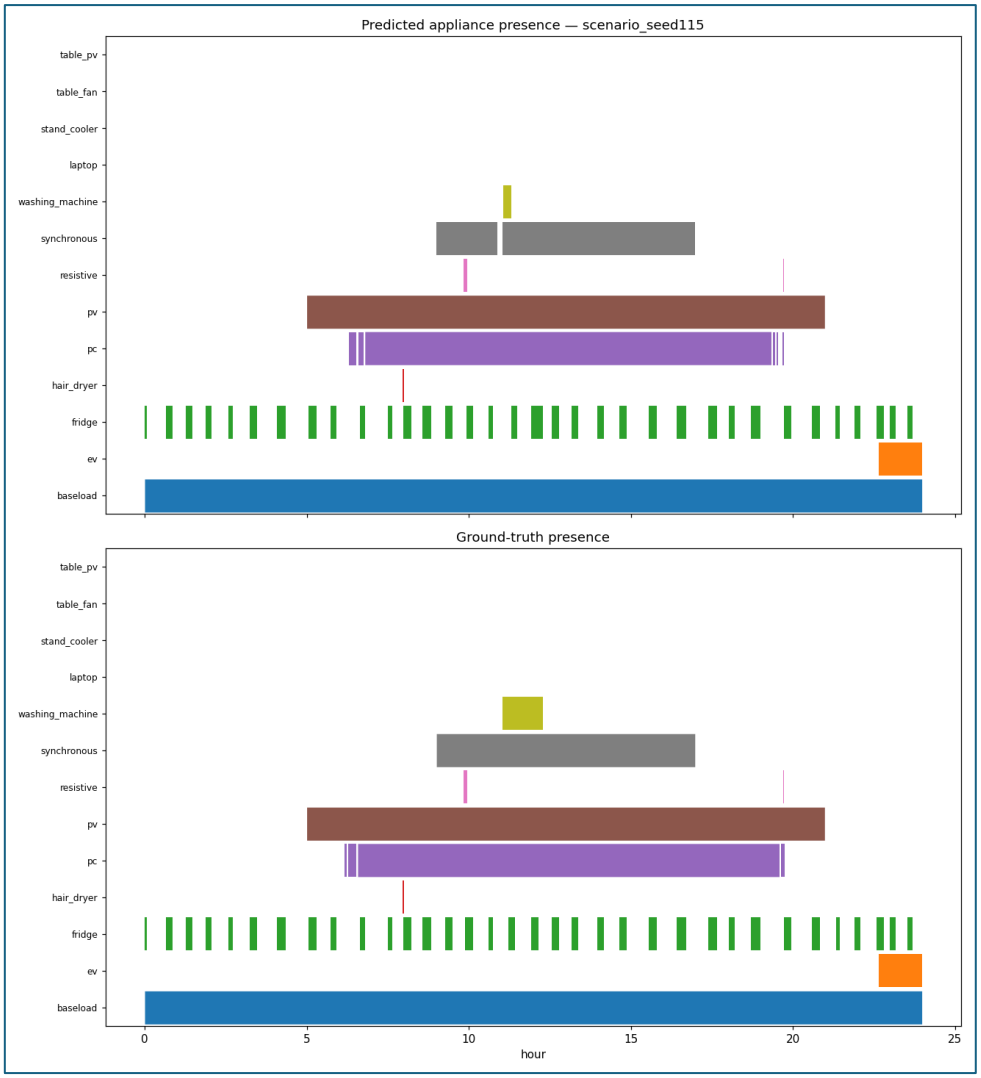

Figure 4. *Presence on a held-out synthetic corpus scenario (scenario_seed115) - predicted (top) vs ground-truth (bottom) appliance activity.*

6. Milestone 3 readiness and next steps

- **Harmonics on the real meter.** Verify the PAC4200 harmonic-current register file numbers against a live load (they currently read zero), unlocking harmonic features and THD_I on real data.
- **Threshold tuning.** Parameterise the 15 W presence threshold so low-power appliances such as the table fan are detected.
- **More data.** Record several sessions per appliance and longer runs, enabling grouped, leakage-free held-out evaluation.
- **Temporal models.** Add a PyTorch CNN/LSTM for full cycle-length sequence modelling; the current MLP is the no-dependency neural baseline.

7. Documentation and repository

The plan and the as-built description are committed to the repository: *Docs/06_milestone2_plan.md* (plan) and *Docs/07_ms2_pipeline.md* (what was built), alongside the Milestone 1 component specifications. All source lives under *Scripts/MS2_Pipeline/* (pipeline, training, inference, UI), *Scripts/Aggregator/* (the measured-scenario mixer), and *Scripts/PAC4200_reader/* (the meter reader and recordings).